COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	08/28/2026	**DATE SUBMITTED:**	08/28/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 6 Searching Techniques

1. PROCEDURE OUTPUT

What procedures did you perform?

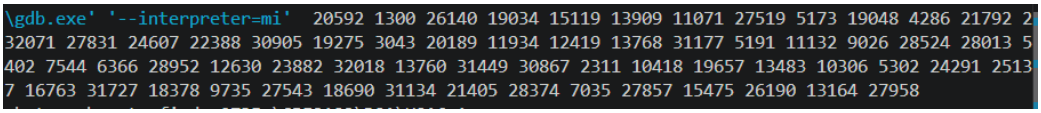
Screenshot	
Observation	These numbers are generated using srand(time(0)) as a random number generator to create a random set of numbers. When observing these random numbers generated, they mostly range from three to five digits, which are very large. Although these numbers are seemingly random, they are still not a true random number because rand() on its own generates pseudo-random numbers using its algorithms and formulas. time() is used to create a random seed for the srand() seed random function to create unique sets of numbers for every execution.

Table 6-1. Data Generated and Observations.

Code

Header Function

```
template <typename T>
void linearSearchAR(int size, T dataFind, T array[]){

    //for loop implementation
    for(int j = 0; j < size ; j++){
        if(dataFind == array[j]){
            std::cout<<"Searching is successful\n";
            return;
        }
    }
    std::cout<<"Searching is unsuccessful\n";
}
```

Main

```
int dataset[max_size];
srand(time(0));
for(int i = 0; i < max_size ; i++){
    dataset[i] = rand();
    std::cout<< dataset[i] << " ";
}
std::cout<<std::endl;

std::cout<< "what number to find: ";
int choice;

std::cin >> choice;

linearSearchAR(max_size, choice, dataset);
```

Output	32176 11332 6436 5982 523 13859 4338 24820 16017 12289 9035 32587 25507 0 31322 25111 28102 2675 7150 31975 30192 25191 21496 24920 15924 22295 what number to find: 22295 Searching is successful
Observation	This is the linear search algorithm used to find a specific value in an array of numbers. The algorithm uses the size, the data or element to find, and the array itself, so its loop can search through the array. Once the condition where the data to find matches the array index, the program will print a message that the search is successful and stop the iteration, or print a message that the search is unsuccessful. It compares each element to the data to find until it satisfies the condition of a data in the array matching to the data to find. So, the best case scenario is O(1) when the data to search is at the first index. If it is at the last index or does not exist, it has the worst case scenario of O(N).

Table 6-2a. Linear Search for Arrays

Code	Header Function
	`void linearSearchLL(Node<T> *nodes, T dataFind){`
	`  // repeat the loop until the current node is not NULL`
	`  while(nodes != nullptr){`
	`  // check if the item = to the current node data`
	`  if(nodes->data == dataFind){`
	`  std::cout << "The data: " << nodes->data << " was found successfully.\n";`
	`  return;`
	`  }`
	`  // move on the next node`
	`  nodes = nodes->next;`
	`  }`
	`  // display unsuccessful`
	`  std::cout << "Searching is unsuccessful.\n";`
	`}`
	Main
	#1
	`// CREATE A NODE`
	`Node<char> *char1 = new_node('J');`
	`Node<char> *char2 = new_node('P');`
	`Node<char> *char3 = new_node('C');`
	`Node<char> *char4 = new_node('L');`
	`// CONNECT THE NODES`
	`char1->next = char2;`
	`char2->next = char3;`
	`char3->next = char4;`
	`linearSearchLL(char1, 'C');`

Output	The data: C was found successfully.
Observation	This is the same linear search algorithm used in the arrays method but this time, it is

	implemented using linked lists, which uses nodes and pointers to perform the iterations to traverse the data stored in the nodes. This iteration continues searching through each node until the data to find matches to the nodes or the iteration reaches nullptr. So, this time complexity is O(N) due to its non-contiguous data structure.

Table 6-2b. Linear Search for Linked List

Code	Header Function:
	`template <typename T>`
	`void binarySearchArr(T data[], int arrSize, T dataFind){`
	`  int low = 0, high = arrSize - 1, middle;`
	`  while(low<= high){`
	`  middle = (int)((low+high)/2);`
	`  if(dataFind == data[middle]){`
	`  std::cout<<"the data was found\n";`
	`  return;`
	`  }`
	`  else if( dataFind < data[middle]){`
	`  high = middle -1;`
	`  }`
	`  else{`
	`  low = middle+1;`
	`  }`
	`  }`
	`  std::cout<<"the data was not found\n";`
	`}`
	
	#1
	`int newArr[10] = {1,2,3,4,5,6,7,8,9,10};`
	`for(int i = 0; i < max_size; i++){`
	`  newArr[i] = i;`
	`}`
	`binarySearchArr(newArr, max_size, 3);`
	#2
	`binarySearchArr(newArr, max_size, 100);`

Output	#1
	`the data was found`
	#2
	`the data was not found`

Observation	The output demonstrates binary search to find the specified element in the array. The algorithm halves the iterations it takes to find the specified element by checking the middle element and looking for one side of the half of the array based on whether the number is greater or less than the middle value. If the value to find is smaller than the middle value, it will iterate through the lower half of the list, as shown in the codes. So, this time complexity has a best case scenario of O(1) if the value to find matches the middle value.

Table 6-3a. Binary Search for Arrays

Code	Header Function:
	`// middle function getter:`
	`template <typename T>`
	`Node<T>* getMiddle(Node<T> *start, Node<T>* last){`
	`    Node<T>* slow = start;`
	`    Node<T>* fast = start;`
	`    // continue until fast reaches the end or last node of the current search range`
	`    while(fast != last && fast->next != last){`
	`        // move slow one step`
	`        slow = slow -> next;`
	`        // move fast two steps`
	`        fast = fast->next->next;`
	`    }`
	`    return slow;`
	`}`

```
void binarySearchLL(Node<T>* start, T dataFind){
    //step 1: set start to the head of the list and set last to null
    Node<T>* last = nullptr;

    while(start != last){
        //Middle element is calculated using the two pointers
        approach discussed above.
        Node<T> *middle = getMiddle(start, last);

        //If the middle element is same as the key to be searched,
        we return it.
        if(middle->data == dataFind){
            std::cout<< "searching is Successful\n";
            return;
        }
        //Else if middle element is greater than the key to be
        searched, we have to search is the
        //right side of the singly linked list. So, we set start
        pointer to the next of middle
        //element
        else if(middle->data > dataFind){
            last = middle;
        }
        //Else if middle element is less than the key to be
        searched, we have to search is the left
        //side of the singly linked list. So, we set last pointer to
        the middle element.
        else{
            start = middle->next;
        }
    }

    std::cout<<"Searching is unsuccessful\n";
}
```

#1

```
Node<int> *int1 = new_node(10);
Node<int> *int2 = new_node(11);
Node<int> *int3 = new_node(12);
Node<int> *int4 = new_node(13);
Node<int> *int5 = new_node(14);
Node<int> *int6 = new_node(15);
Node<int> *int7 = new_node(16);

int1->next = int2;
int2->next = int3;
int3->next = int4;
int4->next = int5;
int5->next = int6;
int6->next = int7;


binarySearchLL(int1, 16);
```

#2

```
binarySearchLL(int1, 30);
```

Output

#1

```
searching is Successful
```

#2

```
Searching is unsuccessful
```

Observation

This linked list implementation of binary search uses two pointers, which are slow for one step and fast for two steps, to obtain the middle value of the list. This method is used to work around the limitations of linked lists since it is not capable of instantaneous direct access to a certain element like the middle value. Despite halving the search time with this method, it still has to

	slowly search through the list to get the middle value, and so it is still an O(N) time complexity for both best and worst case scenarios due to the way linked lists are designed.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Code

searchingAlgo.h

```
template <typename T>
void linearSearchAR(int size, T dataFind, T array[]){
    int comparisons = 0;
    //for loop implementation
    for(int j = 0; j < size ; j++){
        comparisons++;
        if(dataFind == array[j]){
            std::cout<<"Searching is successful\n";
            std::cout << "Number of comparisons: " << comparisons << "\n";
            return;
        }
    }
    std::cout<<"Searching is unsuccessful\n";
    std::cout << "Number of comparisons: " << comparisons << "\n";
}
```

```
template <typename T>
void linearSearchLL(Node<T> *nodes, T dataFind){
    int comparisons = 0;
    // repeat the loop until the current node is not NULL
    while(nodes != nullptr){
        comparisons++;
        // check if the item = to the current node data
        if(nodes->data == dataFind){
            std::cout << "The data: " << nodes->data << " was found successfully.\n";
            std::cout << "Number of comparisons: " << comparisons << "\n";
            return;
        }
        // move on the next node
        nodes = nodes->next;

    }
    // display unsuccessful
    std::cout << "Searching is unsuccessful.\n";

    std::cout << "Number of comparisons: " << comparisons << "\n";
}
```

ILOBp1.cpp

<table>
<tr><td></td><td>

```cpp
#include <iostream>
#include "searchingAlgo.h"
#include "node.h"

int main() {
    // ARRAY
    int newArr[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    std::cout << "ARRAY\n";
    linearSearchAR(10, 18, newArr);

    // LINKED LIST
    Node<int> *int1 = new_node(15);
    Node<int> *int2 = new_node(18);
    Node<int> *int3 = new_node(2);
    Node<int> *int4 = new_node(19);
    Node<int> *int5 = new_node(18);
    Node<int> *int6 = new_node(0);
    Node<int> *int7 = new_node(8);
    Node<int> *int8 = new_node(14);
    Node<int> *int9 = new_node(19);
    Node<int> *int10 = new_node(14);
```

```cpp
    // CONNECT THE NODES
    int1->next = int2;
    int2->next = int3;
    int3->next = int4;
    int4->next = int5;
    int5->next = int6;
    int6->next = int7;
    int7->next = int8;
    int8->next = int9;
    int9->next = int10;

    std::cout << "\nLINKED LIST\n";
    linearSearchLL(int1, 18);

    return 0;
}
```

</td></tr>
<tr><td>Output</td><td>

```
ARRAY
Searching is successful
Number of comparisons: 2

LINKED LIST
The data: 18 was found successfully.
Number of comparisons: 2
```

</td></tr>
<tr><td>Answer</td><td>This linear search uses both implementations which are linked lists and arrays. To count the number of comparisons for these algorithms, a counting variable is added to store the number of comparisons counted, which is done by incrementing the variable inside the loop. The outputs show both have the same number of iterations. The use of this additional code is to visualize the amount of iterations performed by the algorithms for comparison.</td></tr>
</table>

Table 7-1. Problem 1 Answer.

<table>
<tr><td>Code</td><td>searchingAlgo.h

```cpp
// COUNT INSTANCES
template <typename T>
int countInstancesAR(int size, T dataFind, T array[]){
    int count = 0;

    for(int i = 0; i < size; i++){
        if(array[i] == dataFind){
            count++;
        }
    }
    return count;
}
```

```cpp
template <typename T>
int countInstancesLL(Node<T> *nodes, T dataFind){
    int count = 0;

    while(nodes != nullptr){
        if(nodes->data == dataFind){
            count++;
        }
        nodes = nodes->next;
    }
    return count;
}
```

ILOBp2.cpp</td></tr>
</table>

<table>
<tr><td></td><td>

```cpp
#include <iostream>
#include "searchingAlgo.h"
#include "node.h"

int main() {
    // ARRAY
    int newArr[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    std::cout << "ARRAY\n";
    int arrCount = countInstancesAR(10, 18, newArr);
    std::cout << "Instances of 18: " << arrCount << "\n";

    // LINKED LIST
    Node<int> *int1 = new_node(15);
    Node<int> *int2 = new_node(18);
    Node<int> *int3 = new_node(2);
    Node<int> *int4 = new_node(19);
    Node<int> *int5 = new_node(18);
    Node<int> *int6 = new_node(0);
    Node<int> *int7 = new_node(8);
    Node<int> *int8 = new_node(14);
    Node<int> *int9 = new_node(19);
    Node<int> *int10 = new_node(14);
```

```cpp
    // CONNECT THE NODES
    int1->next = int2;
    int2->next = int3;
    int3->next = int4;
    int4->next = int5;
    int5->next = int6;
    int6->next = int7;
    int7->next = int8;
    int8->next = int9;
    int9->next = int10;

    std::cout << "\nLINKED LIST\n";
    int llCount = countInstancesLL(int1, 18);
    std::cout << "Instances of 18: " << llCount << "\n";

    return 0;
}
```

</td></tr>
<tr><td>Output</td><td>

```
ARRAY
Instances of 18: 2

LINKED LIST
Instances of 18: 2
```

</td></tr>
<tr><td>Answer</td><td>This solution is a modification of the algorithm that searches the instances of the element of k, which is 18 in this instance. The algorithm is modified by replacing the return command with an incrementer of the count of instances when 18 is found. The resulting answer should be 2.</td></tr>
</table>

Table 7-2. Problem 2 Answer.

Code	searchingAlgo.h

```cpp
// Modified binary array search for display
template <typename T>
void binarySearchArrDisplay(T data[], int arrSize, T dataFind){
    int low = 0, high = arrSize - 1, middle, iteration = 1;
    while(low<= high){
        middle = (low + high)/2;
        std::cout << "Iteration " << iteration << "\n";
        std::cout << "Element: " << data[middle] << "\n";
        std::cout << "Middle: " << middle << "\n";
        std::cout << "Low: " << low << "\n";
        std::cout << "High: " << high << "\n";

        if(dataFind == data[middle]){
            std::cout << "Data was found\n";
            return;
        }
        else if(dataFind < data[middle]){
            high = middle - 1;
        }
        else{
            low = middle + 1;
        }
        iteration++;
        std::cout << "\n";
    }
    std::cout << "Data was not found\n";
}
```

ILOBp3.cpp

```cpp
#include <iostream>
#include "searchingAlgo.h"
#include "node.h"

int main(){
    int newArr[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    std::cout << "BINARY SEARCH\n";
    std::cout << "Searching for 8\n\n";

    binarySearchArrDisplay(newArr, 10, 8);
    return 0;
}
```

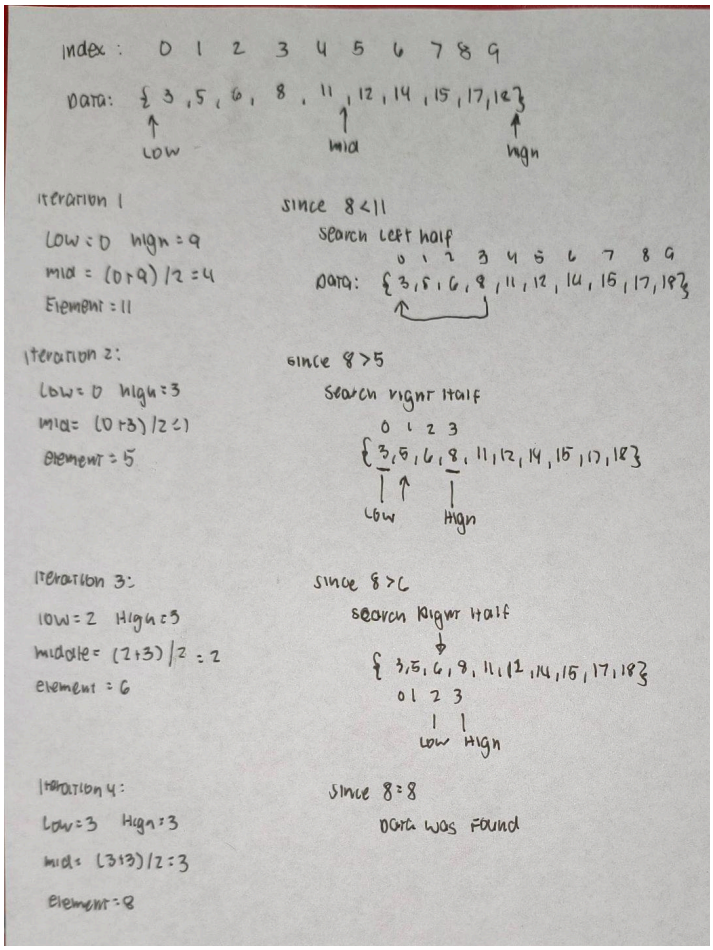
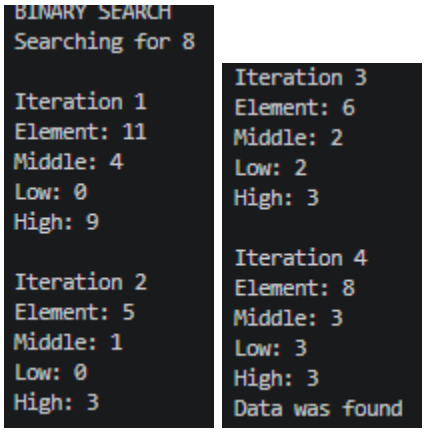
Drawing	
Output	
Answer	The diagram illustrates the process of the binary searching algorithm using arrays. This process halves the array by dividing the low and high values repeatedly until the certain value is found. The diagram shows the halves of each iteration performed by the algorithm to find the number 8.

Table 7-3. Problem 3 Answer.

Code	searchingAlgo.h
	`template <typename T>`
	`void binarySearchRecursive(T data[], int low, int high, T dataFind){`
	`  if(low > high){`
	`  std::cout << "Data was not found\n";`
	`  return;`
	`  }`
	`  int middle = (low + high) / 2;`
	`  std::cout << "Low: " << low << "\n";`
	`  std::cout << "High: " << high << "\n";`
	`  std::cout << "Middle: " << middle << "\n";`
	`  std::cout << "Element: " << data[middle] << "\n\n";`
	`  if(data[middle] == dataFind){`
	`  std::cout << "Data was found\n";`
	`  return;`
	`  }`
	`  else if(dataFind < data[middle]){`
	`  binarySearchRecursive(data, low, middle - 1, dataFind);`
	`  }`
	`  else{`
	`  binarySearchRecursive(data, middle + 1, high, dataFind);`
	`  }`
	`}`

Output	`RECURSIVE BINARY SEARCH`
	`Searching for 8`
	
	`Low: 0`
	`High: 9`
	`Middle: 4`
	`Element: 11`
	
	`Low: 0`
	`High: 3`
	`Middle: 1`
	`Element: 5`
	
	`Low: 2`
	`High: 3`
	`Middle: 2`
	`Element: 6`
	
	`Low: 3`
	`High: 3`
	`Middle: 3`
	`Element: 8`
	
	`Data was found`

Answer	This solution is a modification to the binary searching algorithm to perform a recursive search by calling the function itself to continue the search until a certain value is found. As seen in the code, the function calls itself and moves the middle data if it satisfies a certain condition.

Table 7-4. Problem 4 Answer.

CONCLUSION
This hands-on activity has successfully demonstrated the concepts of two searching algorithms in two implementations, which are the linked list and array implementations of binary search and linear search. The procedures show the basic processes of data searching using the nodes of the linked lists and arrays to find certain values. One particular part of the procedure has also introduced the srand(time()) function, which uses seeds from the time value to generate more random values, even if the random values are not actually random but rather pseudorandom values. The supplementary activities helped show the applications of these procedures by introducing recursive methods, adding counters to visualize the number of iterations, and modifying algorithms to solve certain problems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.